

Personas and the Two-Tier Workspace Model

One Global Workspace produces the programme-level documents. Each EPIC it yields gets its own Mini Workspace, and the same eight personas work in every one of them.

Legend

Reuse as-is

Modify

Build new

KB / platform

External system

PERSONAS - the same eight people appear in every workspace

Product Owner

Owns value and priority

Signs off scope

Business Analyst

Impact analysis, PRD, FRD

Owns the requirement

Scrum Master

EPIC, Feature, Story

Runs the Rally backlog

Architect

ADR, NFR, SDD, SRD

Owns the design

Developer

UI, API and DB code

Builds it

Tester

API, smoke and E2E

Proves it works

UAT

Business acceptance

Accepts or rejects

--all personas--

GLOBAL WORKSPACE - one instance for the programme

Starts from the current KB and works down to a set of EPICs. Nothing here is written per-EPIC.

Impact Analysis

Owner: Business Analyst

Reads the current KB - VB5 and ASP code knowledge, business specs and the MS SQL schema

PRD

Owner: Business Analyst + Product Owner

Product requirements, drawn from the impact analysis

FRD

Owner: Business Analyst

Functional requirements against the MS SQL schema

ADR

Owner: Architect

Architecture decisions - Angular, Spring Boot, Okta, ECS

NFR

Owner: Architect

Performance, availability, security and compliance targets

SDD

Owner: Architect

System Design Document, built from FRD plus ADR plus NFR

SRD

Owner: Architect

System Requirement Document, the input the EPICs come from

LLD and TDD

Owner: Architect/Developer

SDD, SRD, the input the EPICs come from

EPIC set

Owner: Product Owner + Scrum Master

The SRD is decomposed into EPICs. One per Mini Workspace

Architecture takes FRD, ADR and NFR together and produces the SDD and the SRD. The SRD is the only thing that yields EPICs.

SRD yields the EPIC set - one Mini Workspace per EPIC

MINI WORKSPACES - one per EPIC. Ten EPICs means ten Mini Workspaces

EPIC set from the SRD

EPIC 1

gets its own Mini Workspace

EPIC 2

gets its own Mini Workspace

EPIC 3

gets its own Mini Workspace

EPIC N

one more workspace each

Each workspace is scoped to a single EPIC and closes when that EPIC is accepted.

Mini Workspace template - identical in every EPIC

All eight personas join every workspace

Product Owner

Business Analyst

Scrum Master

Architect

Developer

Tester

UAT

Feature

Owner: Scrum Master

EPIC broken into Features in Rally

User Story

Owner: Scrum Master + BA

Stories with 3-factor point estimates

LLD and TDDs

Owner: Architect + Developer

Feature-TDD, UI-TDD, Service-TDD

Coverage Validation

Owner: Business Analyst

SRD to EPIC to Feature to Story chain

UI Code Generation

Owner: Developer

Angular 18 on Axis v18.0.32

API Code Generation

Owner: Developer

Spring Boot controllers, services, repos

MS SQL Integration

Owner: Developer

JPA, JDBC, Flyway, procedure DAOs

Security Review + OPA

Owner: Architect

OWASP, Maven scan, Rego policies

API Testing

Owner: Tester

Python package

UI Smoke Test

Owner: Tester

Component-level smoke checks

UI E2E Testing

Owner: Tester

Python Playwright journeys

UAT

Owner: UAT + PO

Business acceptance, then the pipeline ships it

one workspace per EPIC

Global artifacts

EPIC artifacts

INTEGRATION LAYER - KB FastAPI Service

KB FastAPI Service

REST and OpenAPI for every workspace

Post-generation KB sync hook NEW

Writes each artifact back the moment it is produced

Human approval gate

Draft to Approved, recorded per artifact

GATHER-telemetry v1.4.3

Hook events to the AIG GAME Portal

read / write

write back

approved only

KNOWLEDGE BASE - KB Application: UW CR (VB5 UI + ASP Classic + MS SQL)

Document Knowledge

Specs, BRDs, PRDs

Code Knowledge

VB5 and ASP, embedded and analysed

Schema Knowledge

MS SQL, via GATHER-db NEW

SDLC Artifact Store

Every artifact from both tiers

Amazon Bedrock Knowledge Base

Embeddings and retrieval over the vector store

Amazon RDS PostgreSQL with pgvector

sdlc_artifact: artifact_id, sdlc_stage, epic_id, parent_id, workspace_id

UW Credit Risk - SDLC Pipeline across Global and Mini Workspaces

The Global stage runs once. Everything from Feature downwards repeats inside a Mini Workspace, once per EPIC, and every step writes back to the KB.

Legend

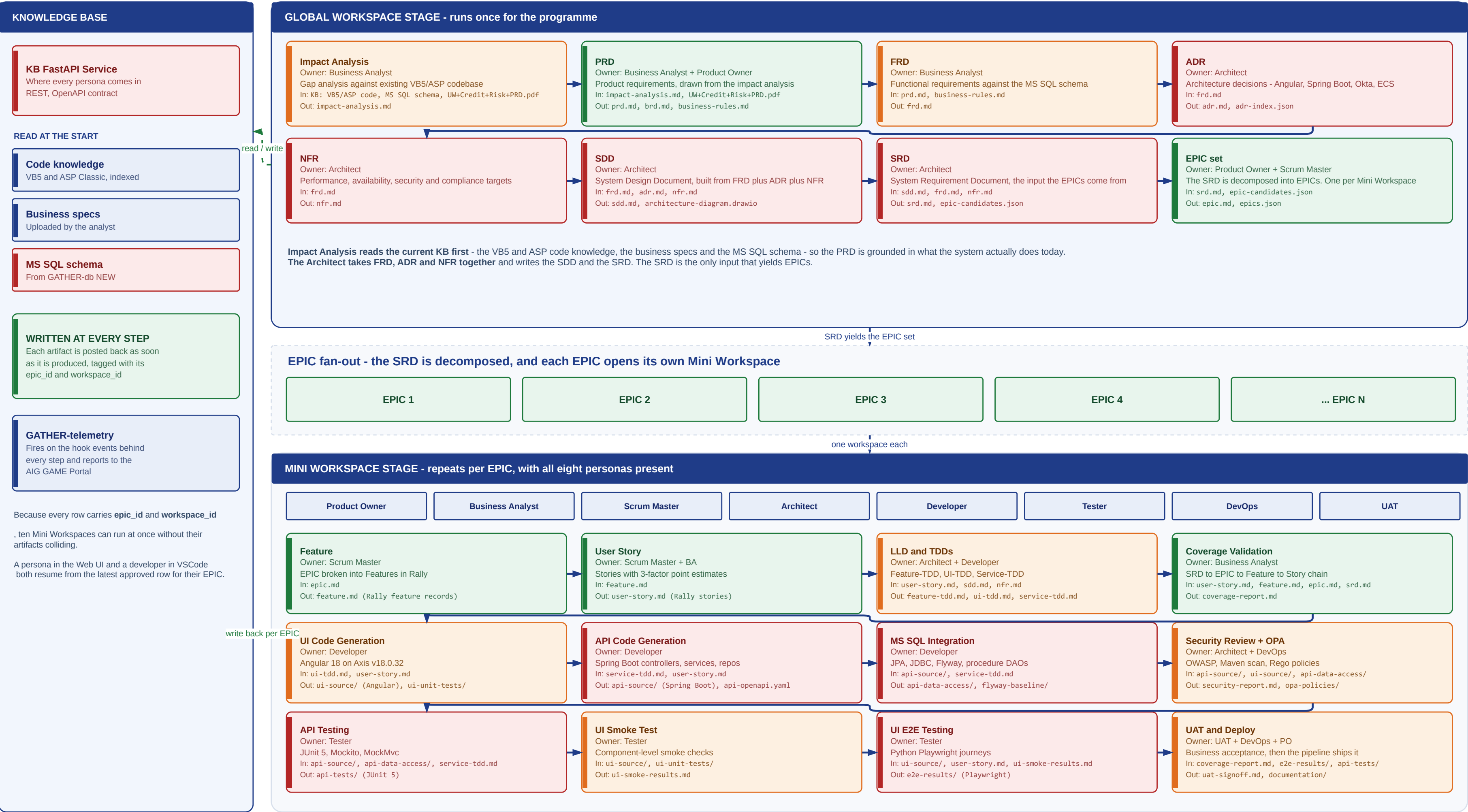
Reuse as-is

Modify

Build new

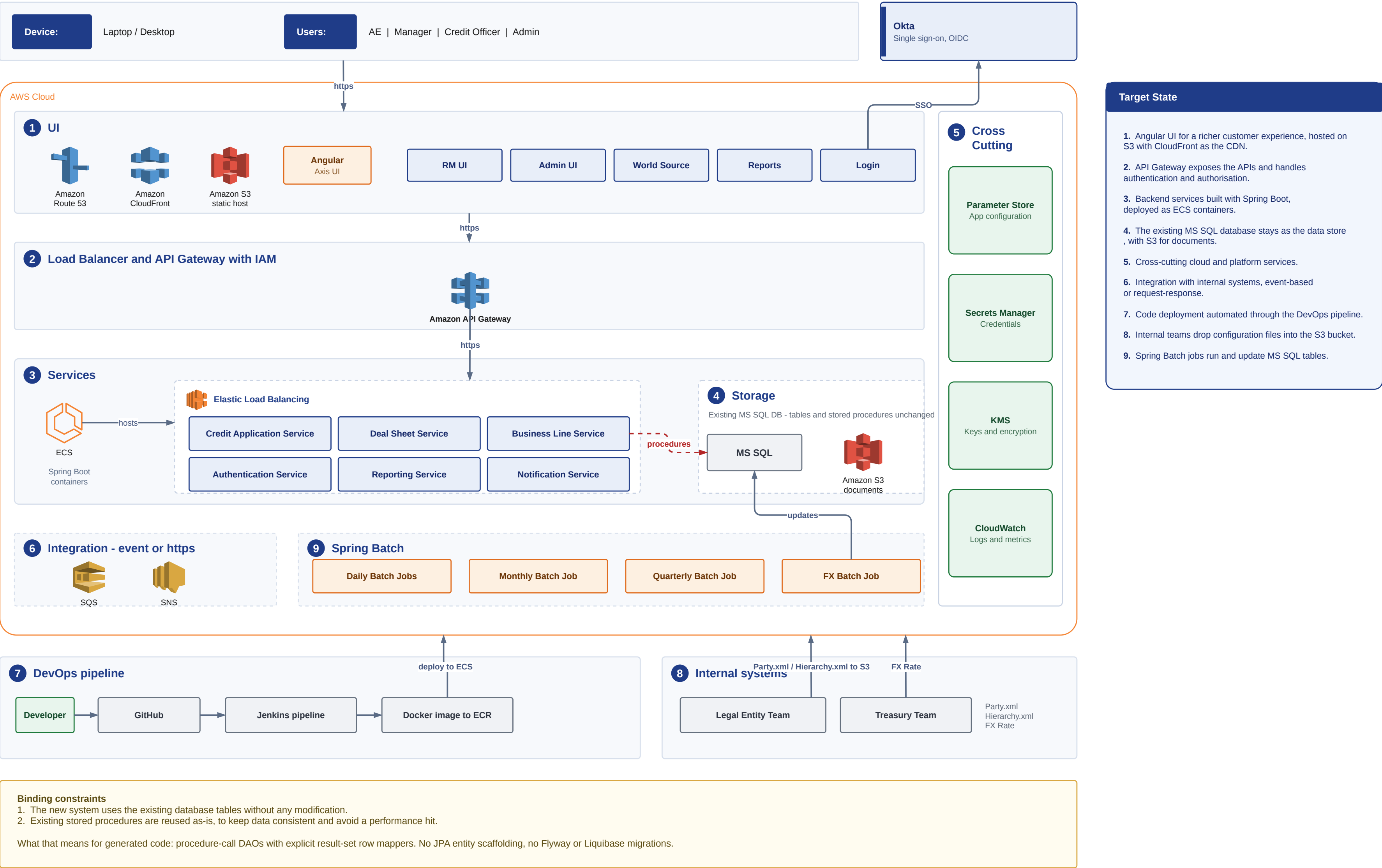
KB / platform

External system



UW Credit Risk - AWS Deployment Overlay

The runtime application plane, with the GenAI SDLC platform that produces it shown separately at the foot of the page.



- Target State
1. Angular UI for a richer customer experience, hosted on S3 with CloudFront as the CDN.

2. API Gateway exposes the APIs and handles authentication and authorisation.

3. Backend services built with Spring Boot, deployed as ECS containers.

4. The existing MS SQL database stays as the data store , with S3 for documents.

5. Cross-cutting cloud and platform services.

6. Integration with internal systems, event-based or request-response.

7. Code deployment automated through the DevOps pipeline.

8. Internal teams drop configuration files into the S3 bucket.

9. Spring Batch jobs run and update MS SQL tables.

GENAI SDLC PLATFORM - the tooling plane that produces the application above

KB FastAPI Service on ECS Fargate
Behind the same ALB, via API Gateway
OpenAPI contract, protected by Okta

Web UI on ECS behind ALB
Okta SSO for both workspaces
Entry point for Global and Hybrid

Amazon Bedrock Knowledge Base
Embeddings and retrieval
over the pgvector store

RDS PostgreSQL with pgvector
sdlc_artifact table
The SDLC state of record

GATHER-telemetry to AIG GAME
Collects Claude Code hook events
and posts them to the GAME API

This plane is separate from the runtime above, which runs on the existing MS SQL database. The FastAPI variant deploys the integration layer as an ECS service, which brings an ALB target group, an API Gateway route, Okta scopes and autoscaling with it. Either way the artifacts land in the same Bedrock and RDS PostgreSQL store, and GATHER-telemetry reports on both.

BAND TO PLUGIN MAPPING - which plugin and which agent builds each numbered band		
Band	Agent	Agents that build it
1 UI	Axis-Agent + Spring boot-Agent	axis-ui-generator, axis-test-generator, tdd-extractor, ui-tdd-designer
2 Load Balancer and API Gateway	Architecture Agent + Spring boot-Agent	architecture-designer, adr-generator, security-config-generator (Okta OIDC)
3 Services - six Spring Boot services	Spring boot-Agent	rest-service-generator, service-layer-generator, exception-handler-generator, springboot-code-validator
4 Storage - existing MS SQL and S3	Spring boot-Agent + Db-Agent	schema-introspector, procedure-introspector, entity-mapper, repository-generator, datasource-config-generator, migration-generator, stored-procedure-dao-generator
5 Cross cutting,Parameter Store Secrets Manager,KMS,CloudWatch	GATHER-security Agent + GATHER-telemetry Agent	crypto-security-reviewer, dependency-security-reviewer (Maven), telemetry hook to AIG GAME
6 Integration - SQS and SNS	Spring boot-Agent	messaging-generator - SQS and SNS listeners and publishers
8 Internal systems - Party.xml, Hierarchy.xml, FX Rate	Axis-Agent + Spring boot-Agent	business-rules-extractor, rego-generator, schema-doc-generator
9 Spring Batch - daily, monthly, quarterly	Spring boot-Agent	spring-batch-generator - Job, Step, ItemReader, ItemWriter

Forward Engineering — Global · Architecture · Mini Workspaces (Three-Tier Pipeline)

LANGGRAPH	CLAUDE
KNOWLEDGE	AWS S3 / RDS
ECS = GLOBAL WS	LAPTOP = MINI WS
LAPTOP = ARCH WS	

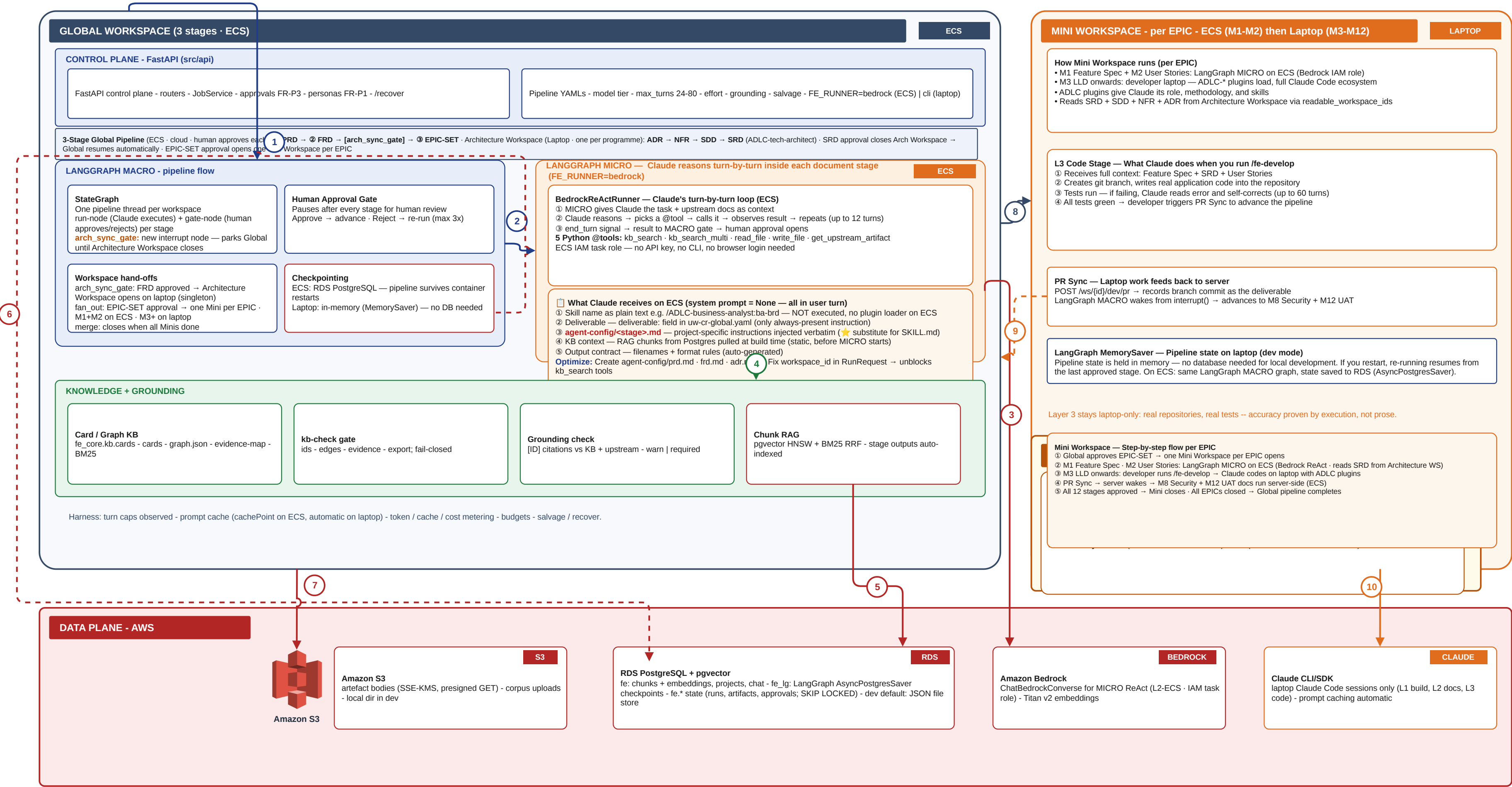
LANGGRAPH AT TWO LEVELS (v2 change)

LANGGRAPH

MACRO — Pipeline routing (graph.py)
StateGraph: one thread per workspace · run-node (Claude) + gate-node (human) per stage
Checkpoint: RDS on ECS · in-memory on laptop

MICRO — Per-stage reasoning loop (bedrock_runner.py · ECS only)
create_react_agent: Claude thinks → calls a tool → observes result → repeats (up to 12 turns)
ChatBedrockConverse: ECS IAM task role — no API key needed · FE_RUNNER=bedrock

ECS: Bedrock IAM task role — no API key or browser login needed
⚡ **LANGGRAPH MICRO -- Plugins are mutually exclusive:**
FE_RUNNER=bedrock → LangGraph MICRO · Boto 3 · Bedrock- claude -Python @tools only
FE_RUNNER=auto/cli → Claude CLI- ADLC Plugins



FLOW KEY 1-10	
① Control plane -> MACRO: orchestrate; approve = Command(resume)	⑥ MACRO checkpointer -> RDS fe_lg (AsyncPostgresSaver; dashed)
② MACRO run node -> MICRO: execute the stage (runner factory: bedrock on ECS cli on laptop)	⑦ Global workspace -> S3: artefact bodies + corpus
③ MICRO -> Bedrock: ChatBedrockConverse turns (IAM task role)	⑧ Global → arch_sync_gate → Architecture WS (laptop ADLC-tech-architect) → SRD → resumes Global
④ MICRO -> Knowledge: artefacts to grounding + kb-check; L1 publish_kb -> Card KB	⑨ Mini -> Global: PR sync via dev/pr -- the gate approves, the graph advances (dashed)
⑤ Chunk RAG -> RDS: chunks + embeddings (auto-indexed after each run)	⑩ Laptop Claude Code -> Anthropic API sessions

LAYER LEGEND — L1/L2/L3 (Bounded ReAct removed · renumbered · no L4 or L5)

L1 — KB Build

One-time laptop setup · ADLC-kb reads RED scan → typed Knowledge Base
kb-check gate validates completeness : ❌ Never on ECS

L2 — Document Stages (ECS + Laptop)

ECS (FE_RUNNER=bedrock): BedrockReActRunner → LangGraph MICRO · 5 Python @tools · IAM role · up to 12 turns
Laptop (FE_RUNNER=auto/cli): CliRunner → Claude Code + ADLC plugins
⚡ MICRO and Plugins mutually exclusive — never both active

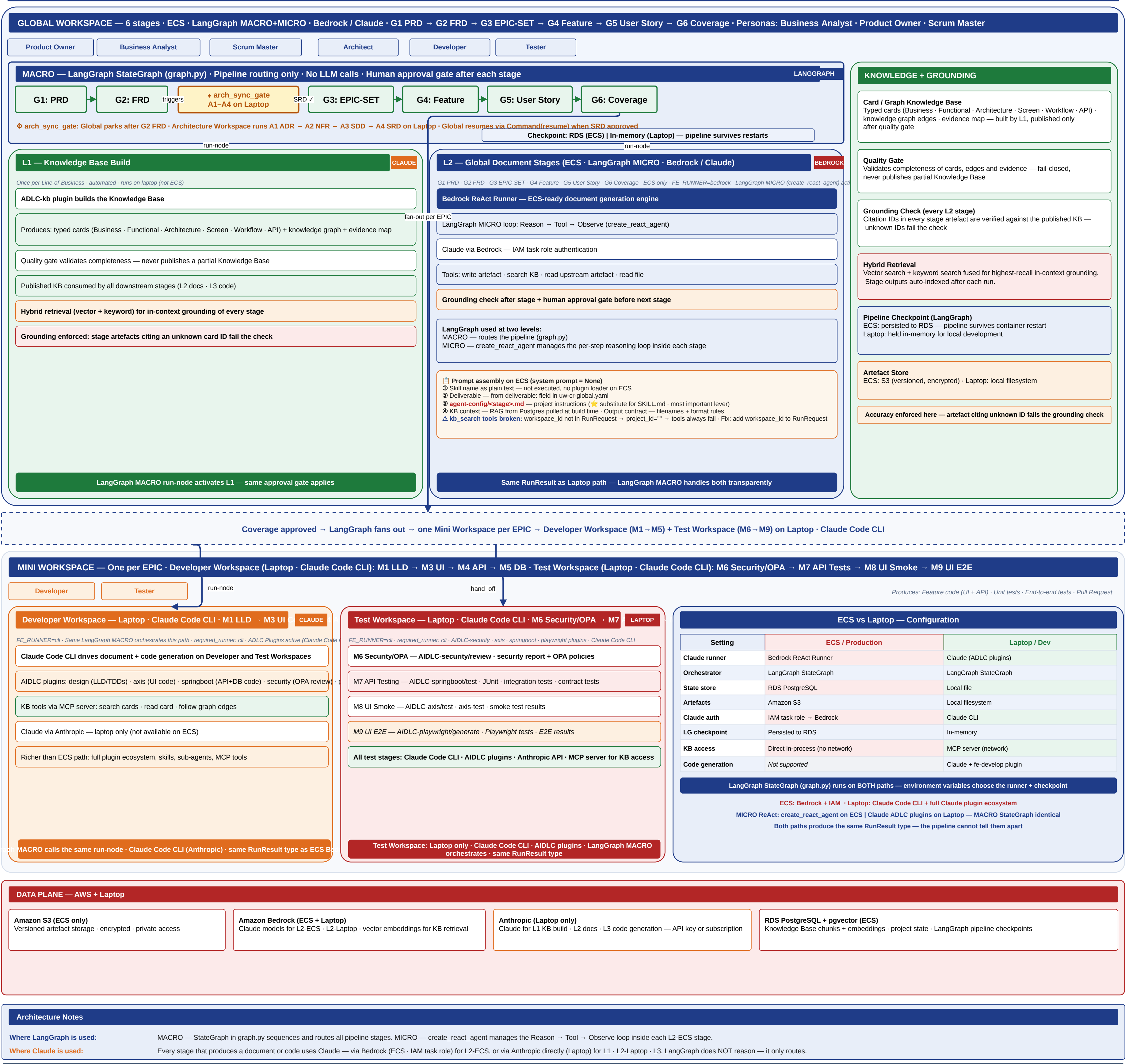
L3 — Code Generation (Laptop only)

LAPTOP_TARGETS = {ui · api · db · tests}
 hand_off pauses server pipeline · /fe-develop → Claude codes + tests + self-corrects (60 turns)
 PR Sync resumes pipeline · ❌ Never on ECS

5 Python @tools — L2 Document Stages · ECS only · FE_RUNNER=bedrock · bedrock_runner.py

kb_search(query, top_k=8) — RAG search on KB chunks via pgvector · one query → top-K results (max 20) · used to ground docs in business rules, screens, workflows
kb_search_multi(queries, top_k=8) — batch KB search · 2–5 queries merged + deduplicated in one call · use when synonyms need separate queries
read_file(filename) — read any existing file from the workspace folder · used to inspect a draft before writing
write_file(filename, content) — write stage deliverable (PRD.md, SDD.md...) to workspace · each call overwrites previous version
get_upstream_artifact(stage_key) — read approved output from a prior stage · e.g. stage_key='frd' → returns approved FRD.md · enables SDD to read FRD + ADP + NFR before writing

Forward Engineering — Global & Local Workspaces



🔧 LAYER LEGEND — L1 / L2 / L3 (Bounded ReAct removed · renumbered · no L4 or L5)		
L1 — KB Build One-time laptop setup · ADLC-kb reads RED scan → typed Knowledge Base kb-check gate validates completeness ·	L2 — Global Document Stages (ECS · Bedrock/Claude · LangGraph MACRO+MICRO) G1 PRD · G2 FRD · G3 EPIC-SET · G4 Feature · G5 User Story · G6 Coverage BedrockReActRunner → create_react_agent · IAM task role · 5 @tools (kb_search, read_file, write_file, get_upstream_artifact)	Developer + Test Workspaces (Laptop · Claude Code CLI) Developer: M1 LLD (AIDLC-design) → M3 UI Code (AIDLC-axis) → M4 API Code (AIDLC-springboot) → M5 DB Integration Test: M6 Security/OPA (AIDLC-security) → M7 API Tests → M8 UI Smoke (AIDLC-axis) → M9 E2E (AIDLC-playwright)

AIDLC Platform — Workspace Architecture & Flow

